

Mid - C++ Merge Sort Code

write me cpp code for fracyional knapsack

Certainly! Here's a simple C++ implementation of the **Fractional Knapsack Problem** using a greedy approach. This version assumes you input the number of items, their weights and values, and the knapsack capacity.

cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Item {
    int value, weight;
};

// Compare two items by value per unit weight
bool cmp(Item a, Item b) {
    double r1 = (double)a.value / a.weight;
    double r2 = (double)b.value / b.weight;
    return r1 > r2;
}

double fractionalKnapsack(int W, vector<Item>& arr) {
    sort(arr.begin(), arr.end(), cmp);

    double totalValue = 0.0;
    for (auto& item : arr) {
        if (W == 0) break;
        if (item.weight <= W) {
            // Take whole item
            totalValue += item.value;
            W -= item.weight;
        } else {
            // Take fraction of item
            totalValue += item.value * ((double)W / item.weight);
        }
    }
}
```

```

        W = 0;
    }
}
return totalValue;
}

int main() {
    int n, W;
    cout << "Enter number of items: ";
    cin >> n;
    vector<Item> arr(n);

    cout << "Enter value and weight of each item:\n";
    for (int i = 0; i < n; ++i) {
        cin >> arr[i].value >> arr[i].weight;
    }
    cout << "Enter knapsack capacity: ";
    cin >> W;

    double maxValue = fractionalKnapsack(W, arr);
    cout << "Maximum value in knapsack = " << maxValue << endl;

    return 0;
}

```

How it works:

- Each item's value/weight ratio is calculated.
- Items are sorted by this ratio (descending).
- The algorithm adds whole items to the knapsack as long as possible, then takes a fraction of the next item to fill the remaining space.

Let me know if you want this code with comments, or a simpler/advanced version!

[go through an example](#)

Absolutely! Let's walk through an example **step by step** using the code above.

Suppose:

- Number of items = 3
 - Values = {60, 100, 120}
 - Weights = {10, 20, 30}
 - Knapsack Capacity (W) = 50
-

Step 1: Calculate value/weight ratio

Item	Value	Weight	Value/Weight
1	60	10	6.0
2	100	20	5.0
3	120	30	4.0

Step 2: Sort items by value/weight (descending)

Order:

- Item 1 (6.0)
 - Item 2 (5.0)
 - Item 3 (4.0)
-

Step 3: Add items greedily

Capacity left: 50

Take Item 1:

- Weight = 10 $\leq$ 50 $\rightarrow$ Take whole item
- Value = 60

- Total value = 60
- Capacity left = $50 - 10 = 40$

Take Item 2:

- Weight = $20 \leq 40 \rightarrow$ Take whole item
- Value = 100
- Total value = $60 + 100 = 160$
- Capacity left = $40 - 20 = 20$

Take Item 3:

- Weight = $30 > 20 \rightarrow$ Take **fraction**
- Fraction taken = $20/30 = 2/3$
- Value taken = $120 \times (20/30) = 120 \times (2/3) = 80$
- Total value = $160 + 80 = 240$
- Capacity left = 0 (knapsack full)

Final Output:

Maximum value in knapsack = 240

How It Happens in Code

If you run the code and enter:

```
3
60 10
100 20
120 30
50
```

Output:

java

Maximum value in knapsack = 240

Let me know if you want a **trace of the code execution** or have any more questions!

write pseudocode

Certainly! Here's **clear and simple pseudocode** for the Fractional Knapsack problem (greedy method):

Fractional Knapsack Pseudocode

sql

Input:

- n items, each with value[i] and weight[i]
- knapsack capacity W

Step 1:

For each item i from 1 to n:

 Compute value_per_weight[i] = value[i] / weight[i]

Step 2:

Sort all items in descending order by value_per_weight

Step 3:

total_value = 0

For each item in sorted order:

 If weight[i] <= W:

 // Take the whole item

 total_value += value[i]

 W -= weight[i]

 Else:

 // Take fractional part of item

```
fraction = W / weight[i]
total_value += value[i] * fraction
W = 0
Break // Knapsack is full
```

Step 4:

Output total_value

Key Points:

- Always pick the highest value/weight ratio first.
- Add items fully if possible; otherwise, take as much as fits.
- Stop when the knapsack is full.

Let me know if you want this with a specific example or want any part explained further!